Arduino UNO Pin Connections:

Component	Pin on Sensor/Mod ule	Connects To Arduino	Notes
DHT11 Sensor	VCC	5V	Power
	GND	GND	Ground
	DATA	D2	Digital data pin (as per your code)
Ultrasonic Sensor (HC-SR04)	VCC	5V	Power
	GND	GND	Ground
	TRIG	D6	Trigger pin
	ECHO	D7	Echo pin
MQ-7 Gas Sensor	VCC	5V	Power
	GND	GND	Ground
	AOUT	A0	Analog signal pin
MPU6050 Accelerometer/Gyro	VCC	5V	Power (can work on 3.3V too)
	GND	GND	Common ground
	SDA	A4	I²C Data line
	SCL	A5	I²C Clock line
LCD (I²C 16x2)	VCC	5V	Power
	GND	GND	Common ground
	SDA	A4	Same I²C Data line
	SCL	A5	Same I²C Clock line
LEDs	Green LED (Anode)	D3	Through 220Ω resistor

	Red LED (Anode)	D4	Through 220Ω resistor
	Both Cathodes	GND	Common ground
Relay Module (for DC Fan)	VCC	5V	Relay Power
	GND	GND	Common ground
	IN	D8	Control signal (LOW → ON, HIGH → OFF as per your requirement)
DC Fan (Load)	+ve Terminal	+12V external supply	External power source
	–ve Terminal	COM terminal of relay	Relay output control
	Relay NO terminal	Connected to ground of power supply	Switches the fan ON/OFF

Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ---------------- PIN DEFINITIONS ----------------
#define DHT11_PIN 2
#define TRIG_PIN 6
#define ECHO_PIN 7
#define RED_LED 4
#define GREEN_LED 3
#define MQ7_PIN A0       // Gas sensor analog pin
#define RELAY_PIN 8      // >>> NEW CODE <<< Relay control pin

LiquidCrystal_I2C lcd(0x27, 16, 2);
uint8_t DHT_Data[5];

// ---------------- SETUP ----------------
void setup() {
  Serial.begin(9600);
  Wire.begin();
  lcd.init();
  lcd.backlight();

  // LED setup
  DDRD |= (1 << RED_LED) | (1 << GREEN_LED);
  PORTD &= ~((1 << RED_LED) | (1 << GREEN_LED));

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  pinMode(MQ7_PIN, INPUT);
  pinMode(RELAY_PIN, OUTPUT);       // >>> NEW CODE <<<
  digitalWrite(RELAY_PIN, HIGH);    // Relay OFF initially (Active LOW)

  initMPU6050();

  lcd.setCursor(0, 0);
  lcd.print("Safety Bot Init");
  delay(2000);
```

```
  lcd.clear();

  Serial.println("==================================================");
  Serial.println(" SAFETY BOT SENSOR MONITOR (TEMP + DIST + TILT + GAS)");
  Serial.println("==================================================");
  Serial.println("Temp\tDist\tTilt\tCO(ppm)\tSTATE\tRELAY");
  Serial.println("--------------------------------------------------");
}

// ---------------- MAIN LOOP ----------------
void loop() {
  int temperature = 0;
  bool dhtOK = readDHT11();
  if (dhtOK) temperature = DHT_Data[2];

  float distance = measureDistance();
  float tilt = getTilt();
  float gasPPM = readGasPPM();

  // ---- Determine status ----
  bool isWarning = false, isCritical = false;
  String reason = "Normal";
  bool fanOn = false;   // >>> NEW CODE <<<

  // Temperature
  if (temperature >= 40 && temperature <= 50) {
    isWarning = true; reason = "Temp";
    fanOn = true; // Turn fan ON in warning range
  }
  else if (temperature > 50) {
    isCritical = true; reason = "Temp";
  }

  // Ultrasonic
  if (distance <= 10 && distance > 5) { isWarning = true; reason = "Dist";
}
  else if (distance <= 5) { isCritical = true; reason = "Dist"; }

  // Tilt
  if (tilt >= 15 && tilt <= 20) { isWarning = true; reason = "Tilt"; }
```

```
else if (tilt > 20) { isCritical = true; reason = "Tilt"; }

// Gas Sensor
if (gasPPM >= 100 && gasPPM <= 200) {
  isWarning = true; reason = "Gas";
  fanOn = true; // Turn fan ON when gas detected
}
else if (gasPPM > 200) {
  isCritical = true; reason = "Gas";
  fanOn = true; // Turn fan ON in critical gas
}

// ---- LED + State ----
String stateMsg = "NORMAL";
if (isCritical) {
  PORTD |= (1 << RED_LED); PORTD &= ~(1 << GREEN_LED);
  stateMsg = "CRITICAL";
} else if (isWarning) {
  PORTD &= ~((1 << RED_LED) | (1 << GREEN_LED));
  stateMsg = "WARNING";
} else {
  PORTD |= (1 << GREEN_LED); PORTD &= ~(1 << RED_LED);
  stateMsg = "NORMAL";
}

// ---- Relay Logic (Fan Control) ----
if (fanOn) {
  digitalWrite(RELAY_PIN, LOW);  // Active LOW → Relay ON
} else {
  digitalWrite(RELAY_PIN, HIGH); // Relay OFF
}

// ---- SERIAL OUTPUT ----
Serial.print(temperature);
Serial.print("\t");
Serial.print(distance, 1);
Serial.print("\t");
Serial.print(tilt, 1);
Serial.print("\t");
Serial.print(gasPPM, 0);
```

```
  Serial.print("\t");
  Serial.print(stateMsg);
  Serial.print("\t");
  Serial.println(fanOn ? "FAN ON" : "FAN OFF");

  // ---- LCD OUTPUT ----
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(stateMsg);
  lcd.setCursor(0, 1);

  if (reason == "Temp") lcd.print("T:" + String(temperature) + "C");
  else if (reason == "Dist") lcd.print("D:" + String(distance, 0) + "cm");
  else if (reason == "Tilt") lcd.print("Tilt:" + String(tilt, 0) +
(char)223);
  else if (reason == "Gas") lcd.print("CO:" + String(gasPPM, 0) + "ppm");
  else lcd.print("All OK");

  delay(1500);
}

// ---------------- DHT11 FUNCTION ----------------
bool readDHT11() {
  uint8_t i, j = 0;
  for (i = 0; i < 5; i++) DHT_Data[i] = 0;

  DDRD |= (1 << DHT11_PIN);
  PORTD &= ~(1 << DHT11_PIN);
  delay(20);
  PORTD |= (1 << DHT11_PIN);
  delayMicroseconds(40);
  DDRD &= ~(1 << DHT11_PIN);

  unsigned long timeout = micros();
  while (((PIND & (1 << DHT11_PIN)) != 0)) if ((micros() - timeout) > 100)
return false;
  while (((PIND & (1 << DHT11_PIN)) == 0)) if ((micros() - timeout) > 200)
return false;
  while (((PIND & (1 << DHT11_PIN)) != 0)) if ((micros() - timeout) > 200)
return false;
```

```
  for (j = 0; j < 5; j++) {
    for (i = 0; i < 8; i++) {
      timeout = micros();
      while (((PIND & (1 << DHT11_PIN)) == 0)) if ((micros() - timeout) >
100) return false;
      unsigned long tStart = micros();
      while (((PIND & (1 << DHT11_PIN)) != 0)) if ((micros() - tStart) >
200) break;
      unsigned long pulseWidth = micros() - tStart;
      if (pulseWidth > 50) DHT_Data[j] |= (1 << (7 - i));
    }
  }
  if ((uint8_t)(DHT_Data[0] + DHT_Data[1] + DHT_Data[2] + DHT_Data[3]) !=
DHT_Data[4])
    return false;
  return true;
}

// ---------------- ULTRASONIC FUNCTION ----------------
float measureDistance() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000);
  return (duration * 0.0343) / 2; // cm
}

// ---------------- MPU6050 FUNCTIONS ----------------
void initMPU6050() {
  Wire.beginTransmission(0x68);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);
}

float getTilt() {
  Wire.beginTransmission(0x68);
```

```
  Wire.write(0x3B);
  Wire.endTransmission(false);
  Wire.requestFrom(0x68, 6, true);

  int16_t AcX = Wire.read() << 8 | Wire.read();
  int16_t AcY = Wire.read() << 8 | Wire.read();
  int16_t AcZ = Wire.read() << 8 | Wire.read();

  float ax = AcX / 16384.0;
  float ay = AcY / 16384.0;
  float az = AcZ / 16384.0;

  float tilt = atan(sqrt(ax * ax + ay * ay) / az) * (180.0 /
3.14159265359);
  return tilt;
}

// ---------------- MQ7 GAS SENSOR ----------------
float readGasPPM() {
  int analogValue = analogRead(MQ7_PIN);
  float ppm = map(analogValue, 0, 1023, 0, 500); // Rough estimate: 0-500
ppm range
  return ppm;
}
```

OUTPUT:

Sensor	Measured Parameter	Trigger Condition	Output
DHT11	Temperature	40–50°C → Warning, >50°C → Critical	Controls LEDs, LCD, Relay for Fan
HC-SR04	Distance	<10cm → Warning, <5cm → Critical	LED + Display update
MPU6050	Tilt angle	15–20° → Warning, >20° → Critical	LED + Display update
MQ-7	Gas ppm	100–200 ppm → Warning, >200 ppm → Critical	LED + Display update + Relay for Fan

OUTPUT:

Sensor	Parameter Range	System State	LED Indication	LCD / Serial Display Message
Temperature (°C)	`< 40°C`	**NORMAL**	🟢 Green ON, 🔴 Red OFF	"Normal – Temp OK" / `T:xx°C`
	`40–50°C`	**WARNING**	🟢 OFF, 🔴 OFF	"Warning – High Temp" / `T:xx°C`
	`> 50°C`	**CRITICAL**	🔴 ON, 🟢 OFF	"Critical – Overheat!" / `T:xx°C`
Distance (cm)	`> 10 cm`	**NORMAL**	🟢 Green ON, 🔴 Red OFF	"Safe Distance" / `D:xxcm`
	`5–10 cm`	**WARNING**	🟢 OFF, 🔴 OFF	"Warning – Obstacle" / `D:xxcm`
	`< 5 cm`	**CRITICAL**	🔴 ON, 🟢 OFF	"Critical – Too Close!" / `D:xxcm`

Tilt (°)	`< 15°`	**NORMAL**	🟢 Green ON, 🔴 Red OFF	“Normal – Stable” / `Tilt:xx°`
	`15–20°`	**WARNING**	🟢 OFF, 🔴 OFF	“Warning – Tilt Detected” / `Tilt:xx°`
	`> 20°`	**CRITICAL**	🔴 ON, 🟢 OFF	“Critical – High Tilt!” / `Tilt:xx°`
CO Gas (ppm)	`< 100 ppm`	**NORMAL**	🟢 Green ON, 🔴 Red OFF	“Air Safe” / `CO:xxppm`
	`100–200 ppm`	**WARNING**	🟢 OFF, 🔴 OFF	“Warning – Gas Leak” / `CO:xxppm`
	`> 200 ppm`	**CRITICAL**	🔴 ON, 🟢 OFF	“Critical – Toxic Gas!” / `CO:xxppm`